Big Data Storage Final Project

MongoDB design for QuickCart Supermarket

Group: group_xxx, replace with the final group number before Moodle submission

Student	Student number
Daniyal Ahmad	20241831
Artemii Alekseev	20240645
Viktoriia German	20240650
João Panizzutti	20241624
Ivan Romanov	20240639

Course deliverable: report, BSON backup, query file, and presentation

One page description

Company chosen: QuickCart Supermarket, a three branch food and household retail chain modelled from the supermarket sales dataset. The business process is the checkout flow: a customer buys one or more products, pays by cash, credit card, or electronic wallet, and leaves a rating.

The source file has 1000 receipts from 2019, three branches, six product lines, three payment methods, and two customer types. Total recorded revenue is $322,966.75; gross income is $15,379.37.

MongoDB fits this dataset because one receipt can be stored as one document. Branch, customer, payment, totals, and line items stay in the same record. This keeps normal checkout queries simple and still lets us group by branch, product, payment, hour, and customer type.

Dataset facts

Model design decisions

The database uses one main collection: sales_receipts. Each receipt embeds branch, customer, payment, financials, and an items array. The source data has one line item per receipt, but the array keeps the model ready for multi item receipts without a second collection.

The model is built around the questions store managers ask first. Most questions start from receipts, so branch city, customer type, payment method, and item details stay inside the receipt. The tradeoff is repeated branch and product text.

The project stays well under the assignment limit of 10 collections. A larger production system could split customers, stock, and product catalogue records, but that would add complexity not needed for this dataset.

Data quality work

I cleaned the source rows before saving them as BSON. Numbers were converted to numeric types, sale date and time became a MongoDB date, category text was trimmed, duplicate invoice ids were removed, branch codes were mapped to the city names used in the source, and financial values were rounded to four decimals.

It also adds sale_month, sale_weekday, and sale_hour fields. Those derived fields keep common management queries simple and readable in Compass.

I also checked row count, unique invoice count, BSON size, sample document types, and the required files before making the final zip.

Validation rules

The validator requires invoice id, branch, customer, item array, sale date, sale month, sale weekday, sale hour, payment, financial values, and rating. It also restricts branch codes to A, B, and C; customer type to Member or Normal; gender to Female or Male; payment method to Cash, Credit card, or Ewallet; product line to the six known lines; quantity to at least one; prices and totals to non negative numbers; sale hour to 0 through 23; and rating to the 0 to 10 interval.

Indexes and performance

Indexes are defined for sale_datetime, branch plus date, product plus total, payment plus date, customer plus product plus rating, rating plus total, and sale hour. The point is not to index everything. Each index maps to one query or aggregation we use in the report.

On this 1000 document course dataset, branch C in March drops from 1000 possible checks to about 106 receipts. High value Food and beverages starts from 174 category receipts and returns 37 above 500. Low rating follow up starts from 153 receipts, not from the whole collection.

The cost is a few extra writes and some index storage. For this project that is okay because inserts are simple, and the report mostly cares about reads.

Business insights from the data

The top product line is Food and beverages with $56,144.84 in sales. The strongest branch is C, Naypyitaw, with $110,568.71 in sales.

Average ticket value is $322.97. Average customer rating is 6.97 out of 10. Member customers generate $164,223.44, slightly more than normal customers at $158,743.31.

The aggregations show branch results, product results, sales by hour, and customer type plus payment mix.

Checked query outputs

Query 1, branch C in March: count 106.

Query 2, high value Food and beverages: count 37.

Query 3, cash above average ticket: count 142.

Query 4, member Sports and travel: count 87.

Query 5, low rating follow up: count 153.

Branch aggregation checked locally: C | $110,568.71 | 328 tickets | avg rating 7.07; A | $106,200.37 | 340 tickets | avg rating 7.03; B | $106,197.67 | 332 tickets | avg rating 6.82

Product aggregation top rows checked locally: Food and beverages | $56,144.84 | 174 tickets | avg rating 7.11; Sports and travel | $55,122.83 | 166 tickets | avg rating 6.92; Electronic accessories | $54,337.53 | 170 tickets | avg rating 6.92; Fashion accessories | $54,305.89 | 178 tickets | avg rating 7.03

Hour aggregation top rows checked locally: hour 19 | $39,699.51 | 113 tickets; hour 13 | $34,723.23 | 103 tickets; hour 10 | $31,421.48 | 101 tickets; hour 15 | $31,179.51 | 102 tickets; hour 14 | $30,828.40 | 83 tickets

Advantages and disadvantages

Advantages: the receipt document matches how the business reads data, query commands stay short, validation protects common mistakes, and indexes match the most repeated questions.

Disadvantages: product and branch values repeat. If a product name changes, many documents need updates. The dataset has one item per receipt, even though the model can hold several items.

The model is good for sales analysis and class work. A real chain could add stock, supplier, promotion, and customer collections if the business needs them.

Top product lines by revenue

Delivery files

group_xxx.bson contains the BSON backup for the sales_receipts collection.

group_xxx.txt contains validators, indexes, queries, aggregations, and explain commands.

group_xxx_presentation.pptx is the 10 minute defense deck.

group_xxx.zip contains the full submission package.